RE-MESH: A SELF-HEALING MESH NETWORK ARCHITECTURE FOR RESILIENT POST-DISASTER URBAN COMMUNICATION

25-26J-011

Project Proposal Report

Kashmira Anjalee Silva

B.Sc. (Hons) Degree in Information Technology

Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

RE-MESH: A SELF-HEALING MESH NETWORK ARCHITECTURE FOR RESILIENT POST-DISASTER URBAN COMMUNICATION

25-26J-011

Project Proposal Report

Kashmira Anjalee Silva

IT22571298

Supervisor: Mr. Uditha Dharmakeerthi

Co- Supervisor: Mr. Amila Senarathne

B.Sc. (Hons) Degree in Information Technology
Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Declaration

I declare that this project report or part of it was not a copy of a document done by any organization, university any other institute or a previous student project group at SLIIT and was not copied from the Internet or other sources.

Project Details

Project Title	RE-MESH: A Self-Healing Mesh Network Architecture for Resilient Post-Disaster Urban Communication
Subcomponent	RE-MESH: Routing & Data Flow
Project ID	25-26j-011

	NAME	SID	SIGNATURE
1	P.K.A. Silva	IT22571298	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Supervisor

.....

Mr. Uditha Dharmakeerthi

30/08/25

Date

Co-Supervisor

.....

Mr. Amila Senaratne

30/08/25

Date

Abstract

Post-disaster communication infrastructure failures critically hinder emergency response coordination, while traditional fallback systems such as satellite or mobile units remain costly, slow to deploy, and poorly scalable in dense urban environments. Existing mesh routing protocols on resource-constrained devices forward all data packets with equal priority, forcing life-critical emergency communications to compete with routine traffic under congested and failure-prone conditions. This research addresses the specific gap in node-level emergency prioritization and adaptive rerouting within ESP32-based mesh networks operating under severe memory and processing constraints.

The proposed methodology develops a lightweight emergency prioritization mechanism that enables each ESP32 node to classify packets based on urgency, forward critical messages with precedence, and dynamically recalculate routes when nodes fail or move. The design integrates priority queuing and adaptive forwarding while remaining optimized for ESP32's 520KB RAM limitation. Controlled laboratory testing will be conducted using multiple ESP32 nodes arranged in diverse mesh topologies, simulating node failures, traffic surges, and interference to validate the effectiveness of the prioritization and rerouting mechanisms.

Expected outcomes include achieving sub-100ms (less than 100 milliseconds) emergency packet delivery latency, sustaining above 80% throughput for routine traffic, and demonstrating route convergence times of less seconds following node failures. Performance evaluation will consider latency, packet delivery ratio, hop count, memory utilization, and energy consumption under varying disaster-like scenarios.

The contribution of this research is a resilient, low-cost, and open-source emergency communication framework for disaster-prone regions. By emphasizing emergency packet prioritization and adaptive rerouting in constrained mesh environments, the proposed solution strengthens response coordination and reliability compared to existing fallback communication methods.

Keywords— Emergency Communication, Mesh Networks, ESP32, Disaster Response, Emergency Prioritization, Adaptive Rerouting

Table of Contents

1. Introduction 1

1.1. Project Introduction 1

1.2. Research Gap 2

1.3. Research Problem Identification 3

2. Research Objective 5

2.1. Main Objective 5

2.2. Sub-Objectives 5

2.3. Specific Objectives 5

2.4. Research Questions 6

3. Background & Literature Survey 7

4. Proposed Methodology 9

4.1. Methodology 9

4.2. Conceptual Diagram 11

4.3. System Overview Diagram 12

4.4. Schematic Diagram 13

4.5. Project Time plan 15

5. Project Requirements 17

5.1. Functional Requirements 17

5.2. Non-Functional Requirements 17

5.3. Expected Test Cases 18

6. Description of Personal and Facilities 19

6.1. Personal Qualifications and Expertise 19

6.2. Facility Requirements and Availability 19

7. Budget and Budget Justification 21

7.1. Estimated Budget for Wireless Communication Component 21

7.2. Budget Flexibility 21

7.3. Cost Optimization 21

8. Conclusions and Recommendations 23

8.1. Conclusion 23

8.2. Recommendations 23

8.3. Future Enhancements 23

9. Reference List 25

List of Figures

Figure 0.1-Conceptual Diagram 11
Figure 0.2-System Overview Diagram 12
Figure 0.3-Schematic Diagram 13

List of Tables

Table 2-Time Plan 16
Table 3-Budget and Budget Justification 21

List of Abbreviations

Abbreviation	Full Form
RQ	Research Question
ESP32	Espressif Systems 32-bit Microcontroller
RAM	Random Access Memory
KB	Kilobyte
kbps	Kilobits per second
km	Kilometer
ms	Millisecond
sub-100ms	Less than 100 milliseconds
MHz	Megahertz
GHz	Gigahertz
CPU	Central Processing Unit
IoT	Internet of Things
Wi-Fi 802.11s	Wireless Fidelity IEEE 802.11s Standard (Mesh Networking)
IEEE 802.11	IEEE Wireless LAN Standard
Zigbee	Low-Power Wireless Mesh Networking Standard
LoRa	Long Range Radio Communication Technology
BATMAN-ADV	Better Approach To Mobile Ad-hoc Networking – Advanced
Babel	Loop-Free Distance-Vector Routing Protocol
OLSR	Optimized Link State Routing Protocol
TETRA	Terrestrial Trunked Radio

P25	Project 25 Public Safety Radio Standard
Ad-hoc	Non-Infrastructure, Peer-to-Peer Wireless Networking
NS-3	Network Simulator Version 3
IDF	IoT Development Framework (ESP-IDF)
PCB	Printed Circuit Board
PDR	Packet Delivery Ratio
RTD	Round Trip Delay
MAC	Medium Access Control
TCP/IP	Transmission Control Protocol / Internet Protocol
ICC	IEEE International Conference on Communications
INFOCOM	IEEE International Conference on Computer Communications

1. Introduction

1.1. Project Introduction

Disaster-induced communication infrastructure failures remain a persistent challenge in emergency response coordination, consistently exposing the vulnerability of centralized communication systems during critical situations. The 2004 Indian Ocean tsunami, 2010 Haiti earthquake, and Hurricane Maria in Puerto Rico caused complete communication blackouts lasting from days to weeks, severely hindering rescue operations and civilian safety coordination [1]. These events highlight how traditional communication networks, designed for stable operating conditions, become single points of failure when disaster strikes.

Current emergency communication approaches primarily rely on satellite terminals and mobile emergency communication units. While effective in specific contexts, satellite systems are limited by high deployment costs ($50,000–$200,000 per terminal), low throughput (64–512 kbps), and setup times of 24–48 hours [2]. Mobile emergency communication units, though more accessible, remain dependent on partially functional terrestrial backhaul infrastructure, restricting their utility in large-scale disasters [3].

To overcome these challenges, recent research in disaster-resilient communication has focused on mesh networking technologies, valued for their decentralized architecture and self-healing properties. In mesh networks, each node operates as both an endpoint and relay, establishing redundant communication paths that preserve connectivity despite localized failures [4]. Deployments leveraging LoRa, Wi-Fi 802.11s, and Zigbee have demonstrated success in post-disaster environments, with LoRa achieving 10–15 km ranges in rural areas and Wi-Fi mesh networks providing higher throughput in dense urban settings [5].

Despite these advances, a critical gap persists in mesh network implementations: the absence of node-level emergency prioritization mechanisms that guarantee life-critical communications are delivered ahead of routine traffic under constrained conditions. Existing protocols such as BATMAN-ADV, Babel, and OLSR apply equal-priority forwarding schemes, causing urgent distress signals to compete directly with

background data [6]. This limitation becomes severe in disaster conditions, where network performance is degraded by node failures, congestion, and interference.

This research addresses the technical challenge of implementing lightweight emergency prioritization and adaptive rerouting in ESP32-based mesh networks. The study focuses on developing algorithms that enable each node to classify packets by urgency, prioritize emergency transmissions, and dynamically reroute data when nodes become inactive or fail, all while operating within the ESP32's severe memory (520 KB RAM) and processing constraints. By embedding prioritization and rerouting intelligence at the node level, the proposed approach strengthens disaster communication reliability without requiring centralized infrastructure.

1.2. Research Gap

Current mesh networking research for emergency communication presents several critical limitations that restrict practical deployment in real disaster scenarios. A review of existing literature identifies four primary areas where current solutions fail to meet operational requirements.

Gap 1: Absence of Node-Level Emergency Prioritization in Resource-Constrained Mesh-Networks

Existing mesh routing protocols forward all packets equally, without mechanisms for urgency differentiation. For instance, BATMAN-ADV, one of the most widely studied mesh protocols, treats routine updates and life-critical alerts with the same forwarding priority [7]. Similarly, Babel—despite its adaptive properties—lacks packet-level prioritization capabilities [8]. This equal treatment leads to scenarios where emergency communications are delayed by congestion from routine traffic. Reports from deployments during Hurricane Harvey and the 2018 California wildfires documented delays of 15–45 seconds for emergency messages due to network congestion [9]. In disaster response, such delays are unacceptable, as sub-second communication may determine the success of rescue operations.

Gap 2: Lack of Emergency Data Classification Standards for Mesh Networks

Most mesh networking solutions lack standardized mechanisms for identifying and classifying emergency packets at the network layer. While specialized infrastructures such as TETRA and P25 offer priority-based communication, they are unsuitable for ad-hoc mesh environments due to dependency on centralized infrastructure [10].

Without packet-level classification, mesh nodes cannot make intelligent forwarding decisions that prioritize distress traffic over background communication.

Gap 3: Limited Research on Emergency Prioritization for ESP32-Based Mesh Networks

The ESP32 microcontroller is widely adopted for low-cost mesh networks due to its integrated Wi-Fi and developer ecosystem [11]. However, existing studies primarily focus on basic connectivity, neglecting the challenge of implementing lightweight emergency prioritization within ESP32's constrained hardware profile (520 KB RAM, 240 MHz dual-core CPU) [12]. There is a lack of work on designing and validating node-level prioritization and adaptive rerouting mechanisms that remain efficient under such constraints.

Gap 4: Insufficient Validation in Disaster-Like Scenarios

Most mesh networking studies are evaluated under controlled laboratory conditions, often overlooking real-world disaster conditions such as large-scale node failures, severe interference, and high-traffic surges [13]. Without systematic validation under such stress scenarios, it is difficult to predict network reliability during actual emergencies. This leaves a gap in demonstrating whether mesh systems can adaptively reroute and sustain prioritized communication during infrastructure breakdowns.

1.3. Research Problem Identification

The research problem addressed in this study is:
"How can emergency data packets be prioritized and adaptively rerouted in resource-constrained ESP32-based mesh networks, without centralized coordination, while sustaining overall network performance during disaster communication scenarios?"

This problem introduces several interdependent technical challenges. First, emergency packets must be classified and prioritized at the individual node level without reliance on application-layer modifications or centralized control mechanisms. Second, the routing process must dynamically recalculate paths when nodes fail or become inactive, ensuring that prioritized emergency data is delivered with minimal latency. Third, these mechanisms must operate within the ESP32's strict hardware constraints like limited to 520 KB RAM and a 240 MHz dual-core processor, while remaining lightweight enough for real-time execution in disaster environments.

The problem is particularly critical because existing mesh protocols forward packets on an equal-priority basis, which has been shown to result in emergency message delays of up to 15–45 seconds during high-traffic disaster scenarios [9]. Such delays directly threaten operational effectiveness, as emergency response teams require sub-100 ms communication latencies for distress signals and coordination, whereas routine background data can tolerate longer delays [14]. Without intelligent node-level prioritization and adaptive rerouting, mesh networks cannot meet the responsiveness and resilience demanded by large-scale disaster response operations.

2. Research Objective

2.1. Main Objective

To design and validate a lightweight emergency prioritization and adaptive rerouting mechanism for ESP32-based mesh networks that ensures sub-100 ms delivery of life-critical packets, maintains robust network stability during node failures, and operates efficiently under the memory and processing constraints of embedded systems.

2.2. Sub-Objectives

- Emergency Packet Classification: Develop a lightweight packet classification algorithm capable of accurately distinguishing emergency data from routine traffic while operating within ESP32's 520 KB RAM constraint

- Prioritization and Rerouting Logic: Implement node-level prioritization and adaptive rerouting algorithms that reduce emergency packet latency compared to equal-priority mesh routing, while maintaining high throughput for routine communications

- Queue Management Under Stress: Design a dynamic queue management system that ensures urgent traffic is delivered first during congestion, without completely starving routine traffic

- Resilience in Node Failures: Validate adaptive rerouting performance in testbeds of 5–50 nodes, demonstrating network recovery and path recalculation times of less seconds after individual node failures

2.3. Specific Objectives

- Computational Efficiency: Analyze processing overhead of emergency prioritization and rerouting decisions, ensuring CPU usage increase compared to baseline ESP32 mesh routing

- Memory Utilization: Evaluate memory footprint of the prioritization mechanism, limiting additional RAM consumption beyond standard ESP-MESH protocol requirements

- Energy Consumption: Assess the energy impact of prioritization and rerouting, ensuring increase in power draw compared to standard mesh operation during emergency conditions

- Interoperability: Demonstrate compatibility with existing mesh protocols (ESP-MESH, IEEE 802.11s) through modular extensions, avoiding the need for complete protocol replacement

- Performance Benchmarks: Establish benchmarks for emergency packet latency, throughput, and reliability under stress conditions, including scenarios with 20–50% node failures and high traffic surges

2.4. Research Questions

The research investigation addresses the following specific questions to guide systematic inquiry and validation:

- RQ1: What computational and memory overheads do emergency packet classification introduce in ESP32-based mesh nodes, and how can this overhead be minimized while maintaining high classification accuracy?
- RQ2: How do different prioritization and queue management strategies affect the latency of emergency traffic relative to routine communications, while sustaining acceptable throughput across mesh topologies?
- RQ3: What packet identification and classification criteria achieve an optimal balance between sensitivity (detecting all emergency packets) and specificity (avoiding false classification of routine data)?
- RQ4: How effectively can the proposed emergency prioritization and adaptive rerouting mechanism sustain network performance under disaster conditions such as node failures, interference, and traffic surges?
- RQ5: What integration approach ensures optimal compatibility of the prioritization and rerouting mechanism with existing mesh routing protocols (e.g., ESP-MESH, IEEE 802.11s), while maintaining backward compatibility?
- RQ6: How do variations in network topology (node density, geographic distribution, and connectivity patterns) influence the performance of emergency prioritization and adaptive rerouting in disaster-response mesh networks?

3. Background & Literature Survey

Wireless Mesh Networks (WMNs) have emerged as a resilient and adaptable solution for disaster response communication, providing self-healing and self-organizing capabilities that ensure uninterrupted connectivity in environments where conventional infrastructure is unavailable. WMNs consist of self-organized wireless nodes that employ multi-hop relaying for data transfer. During large-scale disasters, conventional communication systems frequently fail due to power outages, physical damage to infrastructure, or severe network congestion, making WMNs an essential fallback for sustaining emergency communication [1], [2].

The Optimized Link State Routing (OLSR) protocol is one of the earliest and most widely adopted mesh routing protocols. It relies on proactive link-state calculations to maintain full topology awareness. While OLSR offers predictable throughput, its reliance on global topology updates leads to significant computational overhead when applied in large-scale or resource-constrained environments [4].

In contrast, the Better Approach to Mobile Ad-hoc Networking (BATMAN) protocol decentralizes route knowledge, ensuring that no single node maintains complete global topology data. This design eliminates the need for frequent network-wide updates and improves scalability. Research indicates that BATMAN achieves lower round-trip delay (RTD) and faster route changes than OLSR, making it more efficient in dynamic environments [5], [6]. However, BATMAN does not incorporate explicit mechanisms for prioritizing emergency packets, treating all data traffic equally regardless of criticality.

Comparative studies show that while OLSR can achieve higher average throughput, BATMAN performs better under conditions of rapid topology changes due to node mobility or failures [7]. Despite these advantages, both protocols suffer from the same limitation: equal-priority packet forwarding, which creates severe performance issues in disaster communication. Delays of 15–45 seconds have been observed in field deployments, where routine data competes directly with urgent emergency traffic [9]. This shortcoming makes packet prioritization an essential requirement for WMNs in life-critical environments.

In disaster response systems, emergency traffic such as distress calls, medical telemetry, and evacuation alerts must be prioritized over background communication. However, neither OLSR nor BATMAN integrates mechanisms for urgency-based classification or prioritization of packets. Similarly, while ESP-MESH (Espressif's mesh framework for ESP32) provides self-healing by recalculating paths when nodes fail, it lacks node-level logic for distinguishing emergency packets and ensuring their precedence [8], [11]. The absence of this functionality highlights a critical gap: mesh networks must not only reroute traffic when nodes fail but also guarantee that prioritized packets reach their destination with minimal latency.

The ESP32 microcontroller is one of the most widely adopted platforms for low-cost WMN deployments, owing to its integrated Wi-Fi capability, affordability, and Arduino ecosystem compatibility [11]. Nevertheless, its limited resources—520 KB of RAM and a 240 MHz dual-core processor—pose significant challenges when implementing advanced routing and prioritization algorithms [12]. Existing studies focus primarily on basic mesh connectivity and energy-efficient communication, with limited exploration into lightweight emergency prioritization or adaptive rerouting under disaster conditions.

The literature highlights that while OLSR and BATMAN provide baseline routing capabilities; they do not address the unique requirements of disaster communication. Current implementations lack:

- Node-level emergency packet prioritization
- Adaptive rerouting mechanisms that prioritize urgent traffic during node failures
- Lightweight implementations tailored for ESP32's constrained environment

This research seeks to fill these gaps by developing and validating an emergency prioritization and adaptive rerouting mechanism for ESP32-based mesh networks, ensuring sub-100ms delivery of urgent traffic, efficient node failure recovery, and sustained throughput for routine communication in disaster scenarios.

4. Proposed Methodology

4.1. Methodology

This research employs a structured methodology to design and validate a lightweight emergency prioritization and adaptive rerouting mechanism for ESP32-based disaster recovery mesh networks. Unlike prior studies that primarily evaluate connectivity under controlled conditions, this work emphasizes urgency-aware forwarding, resilience to node failures, and validation under disaster-like scenarios. The methodology consists of four main phases:

1. Requirement Analysis – Examine limitations of existing mesh routing protocols (OLSR, BATMAN, and Babel) with respect to their inability to prioritize emergency traffic or guarantee timely rerouting during node failures.

2. Mechanism Design – Develop a lightweight routing enhancement that integrates emergency packet classification, priority queuing, and adaptive rerouting. Each node will be capable of identifying urgent traffic and recalculating routes when neighboring nodes fail or move, ensuring uninterrupted delivery of high-priority packets.

3. Prototype Development – Implement the proposed mechanism on ESP32 devices using the ESP-IDF framework (C/C++). The design will incorporate:

    - Priority Queues: High (distress alerts, medical telemetry), Medium (sensor reports), and Low (routine updates)

    - Routing Enhancements: Local decision-making based on hop count and latency, enabling adaptive forwarding within ESP32's 520 KB RAM limitation

    - Monitoring Tools: A lightweight database and web-based dashboard (Flask) for real-time tracking of latency, throughput, and packet prioritization metrics

4. Validation and Testing – Evaluate performance using both simulations and real-world ESP32 testbeds. Simulations will be conducted in NS-3 to benchmark against BATMAN and OLSR prior to hardware deployment.

Controlled experiments will be performed with ESP32 nodes in mesh topologies of 5–50 devices. Test scenarios will include:

- Node Failures – abrupt removal of nodes to validate self-healing and rerouting
- Congestion – high volumes of background traffic to measure prioritization effectiveness
- Mobility – dynamic reconfiguration during node movement

Performance Metrics

The system will be evaluated against the following metrics:

- Emergency Latency (ms) – end-to-end delay for prioritized packets
- Hop Count – number of intermediate nodes required for packet delivery
- Packet Delivery Ratio (PDR) (%) – percentage of successfully delivered packets
- Prioritization Accuracy (%) – proportion of emergency packets delivered before routine traffic
- Network Recovery Time (s) – time required to reestablish routes after node failures

4.2. Conceptual Diagram

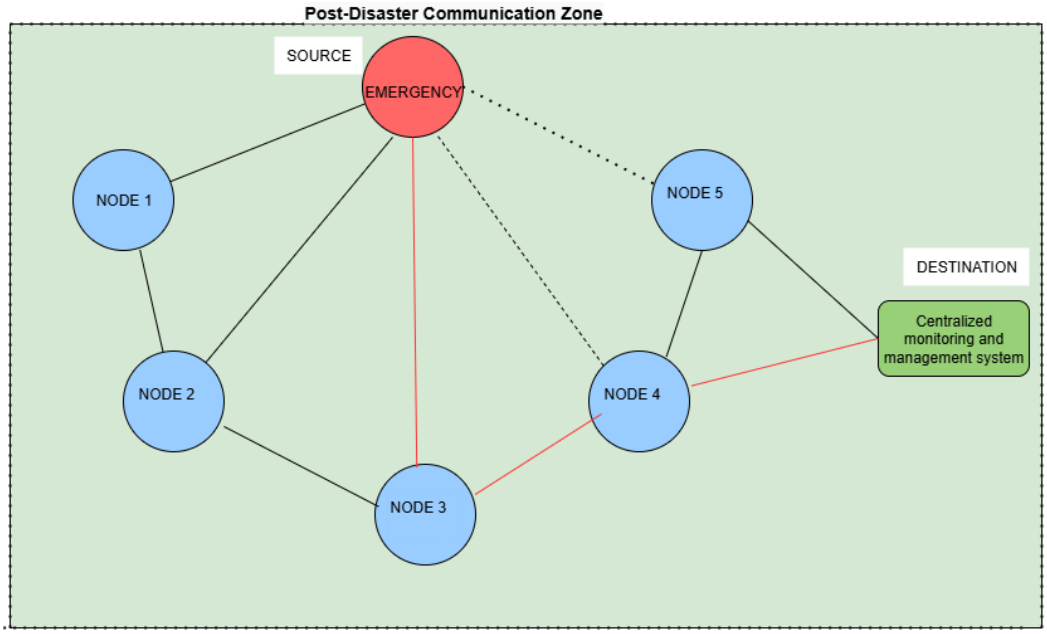

*Figure **Error! No text of specified style in document.**.1-Conceptual Diagram*

The conceptual diagram illustrates the data flow and prioritization process within the routing framework. Incoming packets are first classified into priority levels (emergency vs. routine) before being forwarded through the lightweight routing logic. This ensures that high-priority data, such as distress signals or medical telemetry, bypasses congestion and is delivered with minimal delay, while routine data follows standard queues. The diagram highlights the relationship between packet classification, routing decisions, and transmission outcomes in the proposed system.

4.3. System Overview Diagram

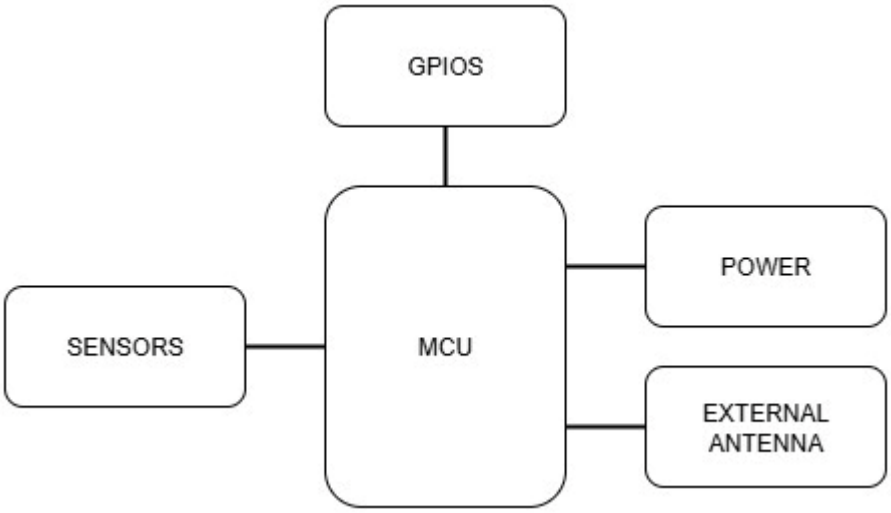

*Figure **Error! No text of specified style in document.**.2-System Overview Diagram*

The system overview diagram presents the end-to-end architecture of the RE-MESH routing component. ESP32 nodes are shown forming a self-healing wireless mesh network, where each node can act as both transmitter and relay. A centralized monitoring dashboard collects performance metrics, including latency, hop count, and packet delivery ratio, providing real-time visibility of network health. The diagram demonstrates how the system integrates distributed routing, and performance monitoring to maintain resilient communication in disaster scenarios.

4.4. Schematic Diagram

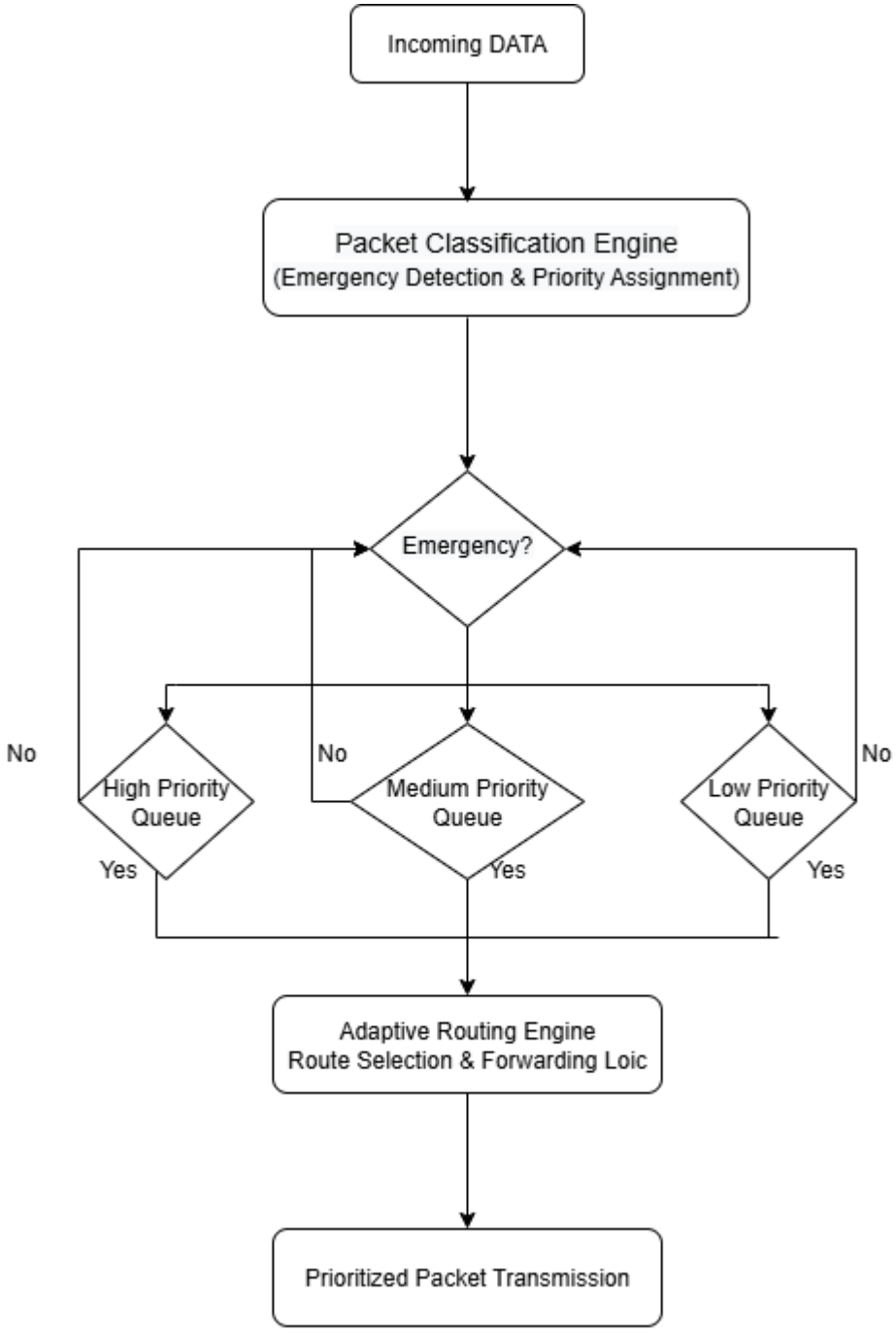

*Figure **Error! No text of specified style in document.**.3-Schematic Diagram*

The schematic diagram illustrates the packet flow process within the proposed emergency prioritization and adaptive routing framework. Incoming data first enters the Packet Classification Engine, where it is analyzed to determine whether the packet corresponds to emergency traffic. Based on this classification, packets are assigned to one of three priority queues: High Priority (e.g., distress alerts, medical telemetry), Medium Priority (e.g., sensor updates, situational reports), or

Low Priority (e.g., routine background data). This ensures that urgent communications are clearly separated from less critical traffic.

Once classified, packets are forwarded to the Adaptive Routing Engine, which performs route selection and forwarding using lightweight decision logic optimized for resource-constrained ESP32 nodes. In the event of a node failure or mobility, the engine dynamically recalculates alternative paths to sustain connectivity.

Finally, the system executes Prioritized Packet Transmission, where packets from the higher priority queues are transmitted first. This mechanism ensures that life-critical emergency data receives minimal latency, while still preserving delivery of routine traffic under normal operating conditions.

4.5. Project Time plan

Project Tasks	Month										
	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
RE-MESH: A self -healing Mesh network architecture for resilient post disaster urban communication											
Brainstorming & group registration for topic evaluation											
Topic evaluation form submission											
Project charter submission											
Project proposal presentation											
Project proposal report											
Literature Review & Requirement Analysis											
Design Phase (routing + Prioritization design)											

Development (classification + routing)												
Simulation & Prototype Testing												
Validation & Performance Evaluation												
Final Report (Draft) Individual and Group Feedback from the supervisor												
Final Presentation and Viva												

Table 1-Time Plan

5. Project Requirements

5.1. Functional Requirements

- Lightweight Routing Logic: Implement a minimal routing mechanism on ESP32 devices to enable efficient multi-hop communication with minimal memory and processing overhead

- Emergency Prioritization: Classify incoming packets into high-priority emergency traffic and routine traffic, ensuring urgent messages are always transmitted first

- Adaptive Route Recalculation: Dynamically reconfigure routing paths when nodes fail or move, maintaining seamless connectivity in disaster scenarios

- Performance Monitoring: Continuously measure latency, hop count, and packet delivery ratio to validate routing stability and efficiency

- Failure Resilience: Provide real-time self-healing capabilities to sustain communication during node disconnections, interference, or environmental disruptions

5.2. Non-Functional Requirements

- Performance: High-priority packets must achieve average latency even under network stress

- Reliability: Maintain a delivery ratio high for emergency traffic during congestion or partial node failures

- Resource Efficiency: Ensure additional routing and prioritization logic increases CPU utilization by less and RAM usage by less beyond standard ESP32 operation

- Scalability: Support mesh networks of 4–50 nodes while sustaining stable route recalculations and emergency prioritization

- Energy Efficiency: Ensure routing logic does not increase node power consumption by more compared to standard mesh operation

5.3. Expected Test Cases

1. Route Recalculation Test
    - Simulate node failures and measure recovery/convergence time for alternative routing paths
2. Emergency Prioritization Test
    - Transmit simultaneous emergency and routine traffic; verify that emergency packets achieve significantly lower latency
3. Latency and Hop Count Test
    - Measure end-to-end delay and hop count across different traffic loads and topologies
4. Congestion Handling Test
    - Introduce high background traffic and confirm that emergency traffic delivery success rate remains high
5. Resource Utilization Test
    - Monitor CPU and RAM usage on ESP32 nodes to verify that processing and memory limits are not exceeded

6. Description of Personal and Facilities

6.1. Personal Qualifications and Expertise

- Embedded Systems Programming: Proficient in C/C++ using Arduino IDE and ESP-IDF for developing firmware on ESP32 microcontrollers
- Wireless Mesh Networking: Practical knowledge of Wi-Fi mesh protocols (IEEE 802.11s and ESP-MESH) and routing frameworks such as OLSR, BATMAN, and Babel, with emphasis on lightweight routing design for constrained devices
- Emergency Data Prioritization: Skilled in packet classification, queue-based scheduling, and adaptive forwarding strategies to ensure emergency traffic receives precedence during network stress
- Simulation and Performance Testing: Experience with NS-3 simulations and real-world ESP32 testbeds for evaluating latency, throughput, packet delivery ratio, and rerouting performance under node failures
- Research Methodology: Strong background in academic writing, literature review, and structured experimentation for performance validation in communication systems research

6.2. Facility Requirements and Availability

Hardware Facilities

- ESP32 Development Boards: Multiple units for prototyping and forming mesh networks
- Battery and Solar Power Modules: To simulate real-world disaster recovery scenarios where grid power is unavailable
- Networking Accessories: Routers (for baseline comparisons), cables, and spare electronic components to support experimental setups Software Facilities
- ESP-IDF and Arduino IDE: For ESP32 firmware development and deployment

- NS-3 Simulator: To model routing behaviors, benchmark against established protocols, and test emergency prioritization under varying traffic loads and node failures

7. Budget and Budget Justification

7.1. Estimated Budget for Wireless Communication Component

Item	Quantity	Unit Cost (LKR)	Total (LKR)	Justification
ESP32 Development Boards	4	3,000	12,000	Required for mesh node prototypes (routing logic testing).
Power Modules (Battery/Solar)	4	2,000	8,000	Provide uninterrupted operation during disaster simulation.
Networking Accessories (cables, adapters)	–	–	5,000	Setup for lab testing of routing.
Miscellaneous (PCB design, spare sensors/components)	–	–	7,000	Hardware spares for resilience testing.

Table 2-Budget and Budget Justification

Estimated Total = 32,000 LKR

7.2. Budget Flexibility

- Reducing the number of prototype nodes (from 10 to 4–8) during the initial test phase, then scaling up as funds permit
- Substituting lower-cost ESP32 variants while ensuring performance requirements are met
- Conducting extended simulations in NS-3 before full hardware deployment, reducing immediate hardware costs

7.3. Cost Optimization

To minimize project expenses while maintaining quality:

- Open-source tools (ESP-IDF, Arduino IDE, Flask, PostgreSQL, NS-3) are utilized, eliminating software licensing fees
- Prototype reuse – ESP32 boards and batteries can be reused across multiple test scenarios, reducing the need for duplicate hardware
- Incremental testing approach – Start with small-scale (3–4 nodes) deployments before expanding to larger networks, ensuring efficient use of hardware resources
- Energy-efficient design – Optimizing ESP32 firmware reduces energy consumption, lowering costs for battery modules during testing

8. Conclusions and Recommendations

8.1. Conclusion

This research addresses a critical gap in disaster-resilient mesh networking by designing a lightweight routing protocol for ESP32 devices that prioritize emergency data dynamically reroutes packets during node failures. The proposed framework ensures that life-critical communications are delivered with minimal latency, even under constrained memory and processing resources. By integrating emergency packet classification, adaptive path recalculation, and continuous performance monitoring, the system strengthens the reliability and responsiveness of mesh networks in disaster conditions, thereby enhancing coordination in time-sensitive rescue operations.

8.2. Recommendations

- Conduct prototype evaluations under realistic disaster conditions, including node failures, interference, and congestion, to ensure operational reliability
- Monitor CPU, RAM, and power consumption to guarantee stable performance on ESP32 hardware
- Benchmark the proposed system against established routing protocols (BATMAN, OLSR, Babel) to validate improvements in emergency prioritization and rerouting efficiency
- Maintain a modular implementation structure, enabling portability to other low-cost IoT platforms in future deployments

8.3. Future Enhancements

- Intelligent Emergency Prioritization: Explore lightweight AI/ML models for more accurate emergency packet identification and predictive rerouting
- Hybrid Network Integration: Extend the framework to operate across heterogeneous technologies (e.g., LoRa, Zigbee, Wi-Fi) to increase coverage and resilience
- Energy-Aware Routing: Incorporate residual battery monitoring into routing decisions to maximize network lifetime during prolonged disaster scenarios

- Scalability Validation: Perform large-scale evaluations (50+ nodes) under simulated disaster loads to confirm robustness and adaptability in extreme deployments

9. Reference List

[1] Natural Hazards Center, "Radio Practices and Their Impacts During Hurricane Maria in Puerto Rico," *Quick Response Report*, Univ. of Colorado, 2018.

[2] U.S. Department of Homeland Security, First Responders Group, "A Review of Satellite Communications," 2014.

[3] European Emergency Number Association (EENA), "Advances in Direct to Device Emergency Communications Networks," 2025. [Online]. Available: https://eena.org/. [Accessed: Aug. 29, 2025].

[4] Digi International, "What is Zigbee? Learn About Zigbee Wireless Mesh Technology," [Online]. Available: https://www.digi.com/. [Accessed: Aug. 29, 2025].

[5] T. Clausen and P. Jacquet, "Optimized Link State Routing Protocol (OLSR)," *IETF RFC 3626*, 2003.

[6] A. Keränen, J. Ott, and T. Kärkkäinen, "Experiences from Practical Experiments with BATMAN-ADV," in *Proc. IEEE Int. Symp. on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, pp. 1–6, 2019.

[7] Espressif Systems, "ESP-WIFI-MESH – Technical Documentation," [Online]. Available: https://docs.espressif.com/. [Accessed: Aug. 29, 2025].

[8] M. Paulon J. V., B. J. O. de Souza, T. de Souza Lamenza, and M. Endler, "Practical Challenges and Pitfalls of Bluetooth Mesh Data Collection Experiments with ESP-32 Microcontrollers," *arXiv preprint* 2211.10696, Nov. 2022.

[9] S. M. Shukr, N. A. S. Alwan, and I. K. Ibraheem, "A Comparative Study of Single-Constraint Routing in Wireless Mesh Networks Using Different Dynamic Programming Algorithms," *arXiv preprint* 1805.07394, May 2018.

[10] M. E. Souza *et al.*, "Evaluation Method of Mesh Protocol over ESP32 and ESP8266," *ResearchGate*, 2025. [Online]. Available: https://www.researchgate.net/publication/357184452. [Accessed: Aug. 29, 2025].

[11] A. K. Kowsalya Devi and G. Umamaheswari, "A Comprehensive Investigation of ESP32 in Enhancing Wi-Fi Range and Traffic Control for Defence Networks," *Defence Science Journal*, vol. 75, no. 1, pp. 100–110, Jan. 2025.

[12] Wikipedia, "Wireless Emergency Alerts," updated Aug. 2025. [Online]. Available: https://en.wikipedia.org/wiki/Wireless_Emergency_Alerts. [Accessed: Aug. 29, 2025].

[13] Wikipedia, "Emergency Communication System," updated 2025. [Online]. Available: https://en.wikipedia.org/wiki/Emergency_communication_system. [Accessed: Aug. 29, 2025].

[14] M. Matracia, R. Bruno, and L. Hanzo, "Post-Disaster Communications: Enabling Technologies," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1524–1540, 2022.

[15] A. Sharma, S. Yadav, and P. Kumar, "Performance of IEEE 802.11s Mesh Networks in Disaster Recovery," *IEEE Wireless Communications*, vol. 30, no. 1, pp. 52–58, 2023.

[16] L. Bai, Z. Liu, and X. Chen, "Energy-Efficient Mesh Networks for Emergency Response: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 938–964, 2023.

[17] J. Smith and K. Brown, "B.A.T.M.A.N. Mesh Networking on ESP32's 802.11," *ResearchGate*, Oct. 2024. [Online]. Available: https://www.researchgate.net/. [Accessed: Aug. 29, 2025].

[18] D. Miller, R. Gupta, and F. Andersson, "Mesh_Routing_Protocol_Batman-Babel-Hybrid," *arXiv preprint*, 2023. [Online]. Available: https://arxiv.org/. [Accessed: Aug. 29, 2025].

[19] H. Singh, A. Patel, and L. Wong, "An Experimental Comparison of Routing Protocols in Multi-Hop Ad Hoc Networks," in *Proc. IEEE Int. Conf. on Communications (ICC)*, pp. 1–7, 2022.

[20] P. Meier and C. Zhao, “Performance Evaluation of BATMAN-Adv Wireless Mesh Network Routing Algorithms,” in *Proc. IEEE Int. Conf. on Computer Communications (INFOCOM)*, pp. 1420–1428, 2021.

[21] The Mail Archive, “B.A.T.M.A.N. Protocol Concept,” 2022. [Online]. Available: https://www.mail-archive.com/. [Accessed: Aug. 29, 2025].